

Smart Lease Summariser

System explainer from the current codebase

What this system does

It helps users turn lease documents into grounded summaries, comparisons, searchable S3 knowledge, and professional chat answers.

Use case

A property or operations user can inspect lease terms without reading every document manually. They can summarise one lease, compare two leases, upload and index S3-backed leases, and ask questions across selected indexed leases while retaining citations and chat history.

Primary users

Property managers, operations teams, lease reviewers, and non-legal stakeholders who need fast lease understanding.

Input documents

Plain text, text-based PDFs, and DOCX files. Scanned PDFs need OCR before the parser can extract text.

Core outputs

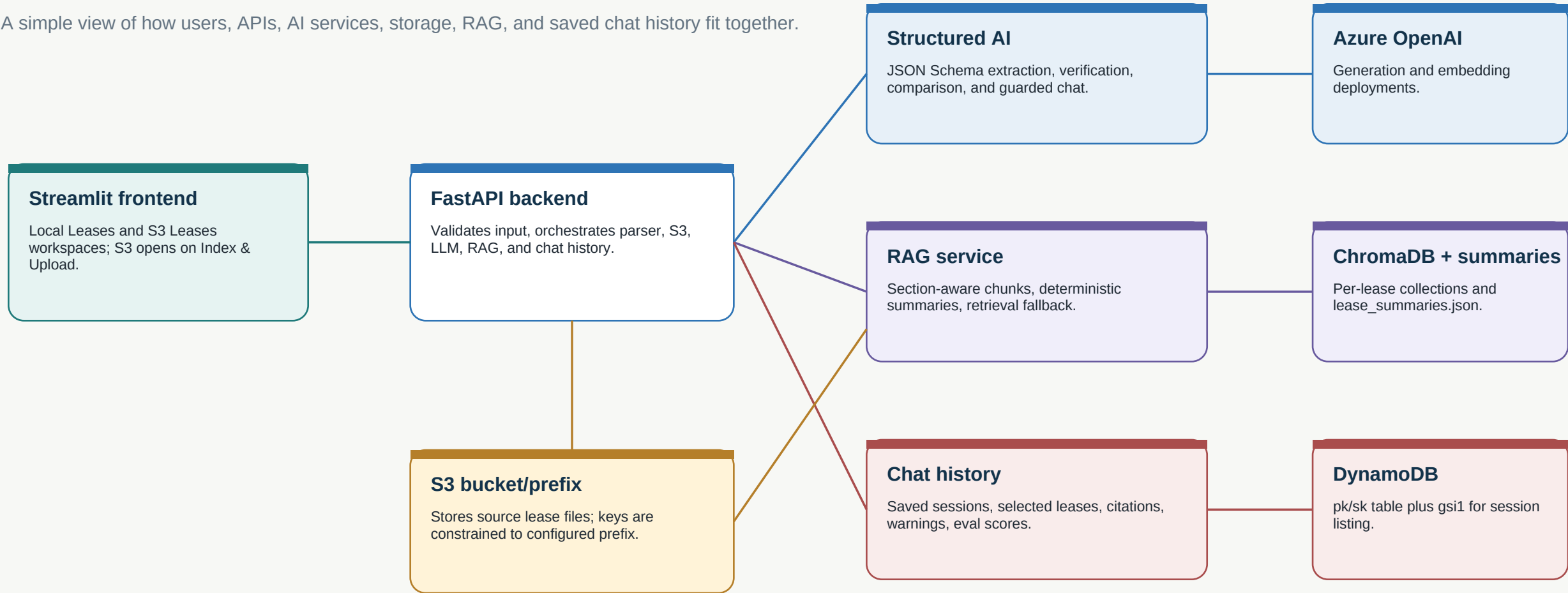
Structured extraction fields, verification checks, comparison differences, indexed search context, chat answers, citations, and warnings.

Source basis

This document is based on app, frontend, scripts, tests, and deployment files. The project specification document was not used.

Stakeholder Architecture

A simple view of how users, APIs, AI services, storage, RAG, and saved chat history fit together.



Key message: local summary/compare stays simple; S3 adds durable files, indexing, search/chat context, and saved sessions.

Overall workings of the system

The application is a two-service product: Streamlit for the user experience and FastAPI for all business logic and integrations.

End-to-end flow

- Users choose either Local Leases or S3 Leases in the Streamlit sidebar.
- Local workflows accept pasted text or uploaded files for summary and comparison.
- S3 workflows list files from one configured bucket/prefix, upload new leases, index existing leases, summarise/compare S3 or indexed leases, and chat against selected indexed leases.
- FastAPI validates inputs, parses files, invokes Azure OpenAI, maps storage errors into API responses, and returns typed Pydantic models to the frontend.

Design principle

The UI stays thin. Validation, parsing, summary, comparison, indexing, retrieval, guardrails, and persistence live in the backend so there is one source of truth for behavior.

Frontend

Streamlit page selector plus tabs. S3 tabs are Index & Upload, Index, Summarise, Compare, and Chat.

Backend

FastAPI endpoints for local files/text, S3 files, indexed leases, RAG indexing, search, chat, chat sessions, and RAG evaluation.

AI services

Azure OpenAI generation calls use strict JSON Schema response formats. Embeddings use a separate deployment.

Storage

S3 holds source files, ChromaDB holds vectors, lease_summaries.json holds extracted facts, and DynamoDB holds chat sessions.

User workflows

The app separates quick document analysis from S3-backed indexed lease work.

Local Leases

- Summarise: paste text or upload one supported file.
- Compare: paste or upload two leases, with mixed input sources supported.
- Both flows enforce the same file parsing and minimum 100-word validation before the LLM pipeline runs.

S3 Leases

- Index & Upload: upload a lease to S3, validate it, then immediately index it into its own collection.
- Index: run background indexing for all supported S3 files and show progress, per-file collection status, chunk counts, and summary counts.
- Summarise and Compare: choose S3 files or indexed lease text. Summarise first returns any stored indexed summary when available.
- Chat: select one or more leases, ask questions in a chat window, toggle sources, save/load/delete sessions, and show RAGAS quality scores when available.

Structured summary and comparison pipeline

RAG does not replace the original structured extraction flow. Summary and comparison remain schema-driven and verified.

Stage	What happens	Why it matters
Parse	Shared parser extracts text from txt, PDF, or DOCX.	One consistent path across local, S3, upload, and indexed flows.
Validate	Unsupported types, empty text, and leases under 100 words are rejected.	Stops poor inputs before model calls and storage writes.
Extract	Azure OpenAI fills LeaseExtraction fields using JSON Schema.	Frontend gets predictable tenant, landlord, address, rent, dates, deposit, notice, obligations,
Verify	A guardrail pass checks every extracted field against source text.	Warnings identify unsupported or missing fields before users rely on output.
Compare	Both leases are summarised and verified first, then structured fields are compared.	Practical differences are based on extracted facts, not raw free-form comparison.

Why JSON Schema

The code builds OpenAI strict JSON schema response formats from Pydantic models. This was chosen so API responses remain predictable, typed, and renderable instead of drifting into model prose.

Public API and data boundaries

The backend exposes separate routes for local, S3, indexed, RAG, and chat-history operations.

Local routes

/summarise, /summarise-text, /compare, and /compare-text parse and validate user-provided content, then run the structured service flow.

S3 routes

/s3/leases lists supported files; /summarise-s3 and /compare-s3 load from S3 but summary can reuse stored indexed summaries.

Indexed routes

/summarise-indexed and /compare-indexed rebuild lease text from Chroma chunks when no stored summary is available.

RAG routes

/rag/index starts background indexing; /rag/index/status reports progress; /rag/search returns matches; /rag/chat answers questions.

Chat sessions

/rag/chat/sessions list, load, and delete saved DynamoDB sessions; /rag/chat creates or appends sessions.

Evaluation

/rag/evaluate and chat-time RAGAS hooks can score faithfulness, answer relevancy, context precision, and context recall when dependencies/config are available.

How and why RAG is used

RAG is used for S3-backed search and chat across indexed leases, not for the core summary pipeline.

Why RAG was added

- Users need to ask questions across many S3 leases rather than opening one file at a time.
- Chat needs relevant source text and prior Q&A context, not only a single one-shot summary.
- Semantic retrieval helps answer detail questions that are not obvious field lookups.
- Stored summaries make aggregate questions deterministic, while retrieved chunks provide evidence and exact wording.

What RAG does not do

- It does not replace structured summary or comparison outputs.
- It does not guess missing lease facts when indexed summaries or retrieved chunks do not support an answer.
- It does not let users change the assistant role into an unprofessional persona.

RAG in one sentence

The system indexes S3 leases into searchable chunks and stored lease summaries, then chat uses summary facts first and retrieved chunks second to produce grounded professional answers.

RAG indexing data flow

Indexing turns each supported S3 lease into two persistent context sources: chunks for retrieval and summaries for deterministic facts.

- Indexing lists supported S3 objects under the configured prefix and clears stale chunks/summaries for that prefix before writing fresh data.
- Each file is downloaded, parsed, validated, section-aware chunked, embedded in batches, and written to ChromaDB.
- Each successfully parsed lease is also sent through the existing structured summary flow and saved to lease_summaries.json with metadata.
- Upload & Index validates one uploaded file, stores it in S3, indexes it into its own per-file collection, and attempts to store a summary immediately.

Output	Stored where	Used for
Lease chunks	ChromaDB per-file collections	Semantic retrieval, citations, and full lease reconstruction.
Lease summaries	lease_summaries.json beside Chroma	Fast facts, aggregate comparisons, and summary citations.
Index status	In-process job state	Progress bar and current file display in the Index tab.
Chat sessions	DynamoDB	Saved shared chat history with messages, citations, warnings, and lease selections.

Chunking strategy

The chunker is hybrid and section-aware: it preserves lease clause context, then splits long sections into retrieval-friendly pieces.

Current strategy

- Detect lease sections line by line using numbered clause headings, uppercase headings, and common lease terms such as rent, deposit, term, repairs, notice, termination, pets, utilities, and special conditions.
- If at least two reliable headings are found, group following text under each detected section.
- Split each section with LangChain RecursiveCharacterTextSplitter using approximately 300 words of text and approximately 50 words of overlap, preferring paragraph, line, sentence, and word boundaries.
- Prefix child chunk text with the section heading, for example 'Section: Rent', and store section_heading, section_index, and section_chunk_index metadata.
- If LangChain text splitting is unavailable, fall back to the original 300-word splitter with 50-word overlap. If headings are not reliable, use the splitter on the whole lease text.

Why this strategy

- Leases are clause-based, so heading context matters for interpreting snippets.
- Small overlapping chunks improve retrieval precision without losing nearby wording.
- Section metadata makes citations easier to understand and preserves reconstruction order.
- A heuristic splitter is cheaper and more predictable than using an LLM during chunking.

Chat answer strategy

Chat combines deterministic summary answers, retrieved snippets, professional guardrails, and saved session state.

Summary-first answers

Questions about tenant, landlord, property address, rent, rent due date, deposit, notice period, obligations, unusual clauses, and summaries can be answered directly from stored summaries.

Deterministic comparisons

Lowest/highest rent or deposit and earliest/latest start or end dates are normalised and compared from summaries, with missing values disclosed.

Retrieval fallback

Other questions embed the prompt and retrieve top chunks from selected leases, or reconstruct lease text from chunks when summaries are missing.

Professional guardrails

The chat system prompt and verification pass reject unsupported facts, legal overreach, and user attempts to force jokes, sarcasm, roleplay, or unprofessional tone.

Sources toggle

Citations include source_type values for summary or chunk, letting the frontend show or hide evidence in the chat window.

Session memory

DynamoDB stores shared sessions so users can load prior messages, selected lease keys, citations, warnings, and timestamps.

Storage, deployment, and configuration

The system uses separate persistence choices for source files, retrieval data, summaries, and chat sessions.

Concern	Implementation	Reason
Source files	AWS S3 bucket and S3_PREFIX	Durable source-of-record for cloud-backed leases.
Vector index	ChromaDB PersistentClient in CHROMA_PERSIST_DIR	Simple local vector store; Docker Compose mounts /app/chroma_db as a volume.
Structured facts	lease_summaries.json beside Chroma data	Deterministic answers without re-running retrieval or LLM calls.
Chat history	DynamoDB table with pk/sk and gsi1	Durable shared sessions across refreshes and container restarts.
Model calls	Azure OpenAI generation and embedding deployments	Separate model roles for structured extraction/chat and embeddings.
Observability	Langfuse traces where configured plus RAGAS scoring hooks	Visibility into model calls and answer quality.

ECS note

In ECS, task roles should provide S3 list/read/write, Secrets Manager read access if settings remain there, DynamoDB session access, and a durable strategy for ChromaDB if tasks are replaced or scaled.

Why key decisions were made

Most decisions optimize for predictable outputs, grounded answers, clear user workflows, and practical deployment.

Strict response schemas

Pydantic models and OpenAI JSON Schema keep responses stable for API clients and frontend rendering.

Verification after extraction

A second model pass checks grounding and creates warnings instead of silently trusting unsupported extraction values.

Separate Local and S3 workspaces

Local users get quick ad hoc tools; S3 users get upload, indexing, summaries, comparison, and chat without cluttering local flows.

Per-file Chroma collections

Selected-lease chat, upload/index, status display, and lease reconstruction are easier when each S3 key maps to its own collection.

Stored summaries during indexing

Aggregate questions such as cheapest rent should use extracted facts rather than depend on which chunks rank highest.

DynamoDB sessions

Chat history needs durability outside Streamlit session state, especially for container restarts and shared app usage.

Current limits

- Scanned PDFs are not OCR'd by the parser.
- Summary accuracy depends on extraction and verification quality.
- Local ChromaDB persistence needs a deliberate production storage strategy for ECS scaling.
- Chat history is shared app-wide because there is no user authentication or tenant isolation in the current codebase.